

Проектний офіс

РЕГЛАМЕНТ ПРОЄКТНОГО ПЛАНУВАННЯ

Узгоджений регламент розробки
програмного забезпечення
та супроводу інформаційних систем

Поточна редакція 26.06.2026

ЛИСТ ЗАТВЕРДЖЕННЯ ТА КОНТРОЛЮ ВЕРСІЙ

Реквізит документа	Значення та відомості
Назва документа	Регламент проектного планування при розробці та супроводі ПЗ ДП «ІСС»
Об'єкт регулювання	Процеси управління та розробки програмного забезпечення в ДП «ІСС»
Версія документа	v1.0.0 (Офіційна редакція)
Дата затвердження	25.06.2026
Статус документа	Затверджено та введено в дію
Відповідність стандартам	ISO 21502:2020, ISO 9001:2015, ISO/IEC/IEEE 12207:2017, ISO/IEC/IEEE 42010:2022, TOGAF
Розробники	Департамент управління проектами та системної архітектури
Погоджено	Архітектурний комітет, Директор з технологій (СТО)
Затвердив	Генеральний директор ДП «ІСС»

Історія змін

Версія	Дата	Автор	Опис внесених змін
0.1.0	28.05.2026	Департамент управління проектами	Первинний проект регламенту планування
0.9.0	15.06.2026	Архітектурний комітет	Додано відповідність TOGAF та ISO/IEC/IEEE 42010
1.0.0	25.06.2026	Робоча група	Затверджено офіційну версію для впровадження

Зміст

Зміст

Анотація	4
Методологічна основа та відповідність аудиту	4
1 Загальна інформація (Purpose & Scope)	5
1.1 Мета документа	5
1.2 Принципи та підходи до розробки	5
1.3 Сфера застосування	5
1.4 Взаємодія з іншими політиками та документами	5
2 Ролі та зони відповідальності (Roles & Responsibilities)	6
2.1 Власник Продукту (Product Owner)	6
2.2 Керівник Проекту з функціями Scrum-майстра (Project Manager / Scrum Master)	6
2.3 Команда Розробки (Development Team)	6
2.4 Бізнес-аналітик (Business Analyst)	7
2.5 Адміністратор JIRA (JIRA Administrator)	7
3 Опис робочих процесів (Workflow Description)	7
3.1 Ієрархія задач і типи робіт у JIRA	7
3.2 Створення й узгодження задач	8
3.3 Пріоритизація задач	8
3.4 Життєвий цикл задач (Workflow)	8
4 Спринт-процес (Sprint Process)	9
4.1 Планування спринту (Sprint Planning)	9
4.2 Виконання спринту (Sprint Execution)	10
4.3 Огляд результатів спринту (Sprint Review / Demo)	10
4.4 Ретроспектива спринту (Sprint Retrospective)	10
5 Реліз-менеджмент (Release Management)	10
5.1 Формування переліку задач для релізу	10
5.2 Задача типу Release Deployment	10
5.3 Процес розгортання та моніторингу	11
6 Управління завданнями та звітність (Reporting)	11
6.1 Відстеження прогресу й трудовитрат	11
6.2 Збирання та візуалізація даних в Apache Superset	11
7 Правила оформлення та ведення документації	12
7.1 Документація вимог (Use Cases / Specifications)	12
7.2 Release Notes	12
7.3 Технічна документація (Technical Architecture)	12
8 Безперервне вдосконалення процесу (Continuous Improvement)	12
8.1 Регулярні ревізії процесу	12
8.2 Процедура внесення змін до Регламенту	13

9	Впровадження регламенту та контроль (Implementation & Controls)	13
10	Пропоновані аналітичні панелі в Apache Superset	13

Анотація

Цей Регламент визначає єдині методологічні засади, життєвий цикл розробки, правила взаємодії учасників проектних команд, норми ведення технічної документації та контролю якості для Державного підприємства «Інформаційні судові системи» (ДП «ІСС»). Метою регламенту є структурування внутрішніх процесів розробки та супроводу програмного забезпечення для підвищення прозорості, мінімізації операційних ризиків та успішного проходження міжнародних технічних і фінансових аудитів.

Методологічна основа та відповідність аудиту

Регламент розроблено на перетині гнучких практик управління (Agile/Scrum) та класичних міжнародних стандартів системної архітектури і управління проектами. Відповідність вимогам міжнародного аудиту забезпечується інтеграцією наступних стандартів:

1. TOGAF (The Open Group Architecture Framework): забезпечує структурування архітектури підприємства за чотирма доменами: бізнес-архітектура, архітектура даних, архітектура додатків та технологічна архітектура. Усі проектні ініціативи (Epic) проходять попередній аналіз на відповідність цільовій архітектурній моделі ICC.
2. ISO/IEC/IEEE 42010:2022 (Systems and software engineering — Architecture description): стандартизує вимоги до опису архітектури систем, виділяючи окремі архітектурні точки зору (views) та забезпечуючи формальну документацію зв'язків між системними компонентами та бізнес-вимогами.
3. ISO/IEC/IEEE 12207:2017 (Systems and software engineering — Software life cycle processes): визначає життєвий цикл програмного забезпечення від формування вимог до виведення з експлуатації. Регламентований у цьому документі workflow у системі JIRA повністю відображає процеси реалізації, інтеграції та верифікації відповідно до цього стандарту.
4. ISO 21502:2020 (Project, programme and portfolio management — Guidance on project management): структурує процеси управління ресурсами, ризиками, змінами та якістю результатів у межах спринтів та релізів.
5. ISO 9001:2015 (Quality management systems): вимагає наявності чітких метрик якості, простежуваності вимог та регулярного аналізу відхилень (наприклад, порівняння запланованого та фактичного часу виконання).

У межах аудиту регламент гарантує розподіл обов'язків (Separation of Duties, SoD):

- Product Owner відповідає за бізнес-вимоги та пріоритети;
- Business Analyst формалізує вимоги та бізнес-правила у вигляді Use Cases;
- Development Team здійснює технічну реалізацію без права самостійної зміни вимог;
- QA/Tester виконує незалежне тестування та підтверджує готовність до релізу;
- Project Manager контролює дотримання регламенту та управляє ризиками;
- JIRA Administrator забезпечує технічну неможливість обходу життєвого циклу завдань.

1. Загальна інформація (Purpose & Scope)

1.1. Мета документа

Цей документ встановлює обов'язковий до виконання регламент розробки нового програмного забезпечення (ПЗ) та супроводу існуючих рішень у межах діяльності ДП «ІСС». Регламент визначає ролі учасників, життєвий цикл завдань у системі трекінгу JIRA, правила часового оцінювання, порядок реліз-менеджменту, правила ведення технічної документації в Confluence/Git та механізми безперервного вдосконалення процесів.

1.2. Принципи та підходи до розробки

Діяльність проектних команд ДП «ІСС» базується на наступних фундаментальних принципах:

- Agile-орієнтація: розробка здійснюється ітеративно-інкрементним шляхом. Це дозволяє оперативно реагувати на зміни нормативно-правового поля, вимог замовників та технологічних стандартів без втрати керованості процесом.
- Адаптований Scrum-процес:
 - Базова тривалість ітерації (спринту) становить 2 тижні.
 - Обов'язковими подіями є: Планування спринту (Sprint Planning), Щоденні наради (Daily Stand-up), Огляд результатів спринту (Sprint Review / Demo) та Ретроспектива спринту (Sprint Retrospective).
- Ітеративність та поетапна доставка цінності: кожна ітерація має завершуватися створенням потенційно працездатного інкременту системи.
- Прозорість (Transparency): поточний стан усіх завдань, оцінки трудовитрат та метрики прогресу є відкритими для учасників процесу та стейкхолдерів через системи JIRA, Confluence та аналітичні дашборди Apache Superset.

1.3. Сфера застосування

Вимоги цього Регламенту є обов'язковими для:

- Усіх довгострокових та короткострокових проектів з розробки, модернізації та супроводу прикладного ПЗ ДП «ІСС».
- Усіх членів внутрішніх команд розробки (штатних працівників) та залучених субпідрядних організацій (вендорів).
- Обов'язкового обліку трудовитрат у годинах для кожної окремої задачі в контурі JIRA.

1.4. Взаємодія з іншими політиками та документами

Цей Регламент діє спільно та узгоджено з Політиками технічних вимог, технічного завдання та технічної документації ІСС, що базуються на стандартах ДСТУ 3973-2000 та ДСТУ 3974-2000. У разі виникнення розбіжностей між цим Регламентом та вимогами внутрішньої інформаційної безпеки (щодо КСЗІ та НД ТЗІ), пріоритет мають вимоги безпеки.

2. Ролі та зони відповідальності (Roles & Responsibilities)

Для забезпечення прозорості та відповідності вимогам зовнішнього аудиту, ролі в проєктах ДП «ІСС» чітко розмежовані. Спільне виконання ролей розробника та тестувальника, або аналітика та аудитора безпеки в межах одного завдання заборонено.

2.1. Власник Продукту (Product Owner)

Власник Продукту є представником бізнес-інтересів замовника (наприклад, ДСА України, органів судової влади) і відповідає за максимізацію цінності продукту.

- Ключові обов'язки:
 - Створення, наповнення та одноосібна пріоритизація Беклогу Продукту (Product Backlog).
 - Визначення бізнес-цілей на кожен спринт та погодження складу завдань під час планування.
 - Тісна співпраця з бізнес-аналітиками для деталізації вимог.
 - Прийняття або відхилення реалізованого функціоналу на Sprint Review відповідно до Критеріїв Прийняття (Acceptance Criteria).
- Результат діяльності: актуальний, прозорий та пріоритизований беклог; чітко сформульовані цілі розробки.

2.2. Керівник Проєкту з функціями Scrum-майстра (Project Manager / Scrum Master)

Роль об'єднує функції класичного управління проєктними обмеженнями (терміни, бюджети, ризики, звіти) та методологічного лідерства (Scrum Master).

- Ключові обов'язки:
 - Організація та фасилітація всіх SCRUM-подій, забезпечення дотримання дисципліни ітерацій.
 - Усунення перешкод (impediments) на шляху команди розробки.
 - Координація зовнішніх залежностей між суміжними командами та системами.
 - Управління ресурсами проєкту, бюджетування та комунікація зі стейкхолдерами.
 - Забезпечення щоденного логування часу командою.
- Результат діяльності: ритмічна робота команди без затримок та перешкод; сформовані релізні плани та дотримання бюджетних лімітів.

2.3. Команда Розробки (Development Team)

Команда є крос-функціональною та містить розробників (Frontend/Backend/Fullstack), тестувальників (QA Manual/Automation), дизайнерів (UI/UX) та технічних письменників.

- Ключові обов'язки:
 - Декомпозиція та технічна оцінка (у годинах) завдань, включених у спринт.
 - Безпосередня розробка, тестування, інтеграція та документування функціоналу.
 - Підтримка актуального стану своїх задач у JIRA протягом дня.
 - Участь у демонстраціях результатів та ретроспективах.
- Результат діяльності: працездатний інкремент продукту в кінці кожного спринту, що повністю відповідає критеріям Definition of Done (DoD).

2.4. Бізнес-аналітик (Business Analyst)

Бізнес-аналітик залучається з пулу аналітиків ДП «ІСС» на вимогу Project Manager для складних інтеграційних чи масштабних функціональних завдань.

- Ключові обов'язки:
 - Збір, аналіз та формалізація вимог від Власника Продукту та стейкхолдерів.
 - Створення специфікацій використання (Use Cases), опис бізнес-правил та побудова BPMN-діаграм процесів.
 - Трансформація вимог у зрозумілі для команди розробки завдання (JIRA Stories/Tasks).
- Результат діяльності: задокументовані Use Cases, BPMN-діаграми та детальні описи бізнес-правил.

2.5. Адміністратор JIRA (JIRA Administrator)

Окрема технічна роль, що забезпечує стабільність та незмінність життєвого циклу розробки.

- Ключові обов'язки:
 - Конфігурування проєктів, типів задач, полів та схем переходів (workflow).
 - Керування правами доступу, забезпечення неможливості ручного переведення задач в обхід обов'язкових статусів (наприклад, перехід з Development відразу в Done без стадії Testing).
 - Налаштування інтеграцій JIRA з Confluence, репозиторіями коду та Apache Superset.

3. Опис робочих процесів (Workflow Description)

3.1. Ієрархія задач і типи робіт у JIRA

Для забезпечення простежуваності (traceability) від стратегічних цілей до конкретного рядка коду використовується наступна ієрархія сутностей у JIRA:

1. Theme (Тема): глобальний стратегічний напрям (наприклад, «Законодавчі зміни 2026», «Міграція на мікросервісну архітектуру»). Реалізується через додавання обов'язкових

терів до Епіків.

2. Епіс (Епік): значний обсяг функціоналу, який не може бути виконаний за один спринт (наприклад, «Модуль виконавчих документів»). Епік має чіткі межі та критерії готовності.
3. Story / Task (Користувацька історія / Задача):
 - Story описує вимогу з точки зору кінцевого користувача та приносить бізнес-цінність.
 - Task описує технічні, інфраструктурні чи дослідницькі роботи (наприклад, «Налаштування індексу бази даних»).
4. Sub-task (Підзадача): технічна декомпозиція Story/Task на конкретні кроки для окремих виконавців (розробка API, верстка інтерфейсу, написання автотестів).
5. Bug (Дефект): опис невідповідності фактичної поведінки системи вимогам технічного завдання.
6. Release Deployment (Розгортання релізу): спеціальний системний тип задачі для керування процесом збирання, тестування на Staging та розгортання на Production.

3.2. Створення й узгодження задач

Будь-яка задача у JIRA починає свій життєвий цикл з опису. Ініціатор задачі (BA, PO або PM) зобов'язаний заповнити:

- Стислий та зрозумілий заголовок.
- Детальний опис (для Stories — за шаблоном: «Як [Роль] я хочу [Дія] для того, щоб [Цінність]»).
- Критерії Прийняття (Acceptance Criteria), за якими тестувальник перевірятиме результат.
- Посилання на пов'язані Use Cases у Confluence або BPMN-діаграми.

Задача вважається узгодженою та переводиться в статус **Ready for Development** тільки після підтвердження Власником Продукту бізнес-цінності та Керівником Проєкту — повноти опису.

3.3. Пріоритизація задач

Пріоритети задач у JIRA розподіляються наступним чином:

- Highest (Блокуючий): критичні дефекти на промисловому середовищі (Production), які зупиняють роботу ключових сервісів, або завдання, невиконання яких загрожує безпеці чи порушує вимоги законодавства з визначеним дедлайном. Розробка інших планових задач припиняється для усунення дефекту.
- High (Високий): важливі планові завдання або дефекти, що мають обхідні шляхи (workarounds), але суттєво впливають на користувачів.
- Medium (Середній): стандартний пріоритет для більшості планових задач та дефектів інтерфейсу.
- Low (Низький): другорядні покращення, рефакторинг або косметичні дефекти.

3.4. Життєвий цикл задач (Workflow)

Для всіх Stories, Tasks та Bugs встановлюється єдиний workflow переходів статусів (див. табл. 3).

Табл. 3: Опис статусів життєвого циклу задач у JIRA

Статус у JIRA	Опис процесу та критерії переходу
To Do	Задача створена, перебуває в беклозі. Очікує первинного аналізу та пріоритизації.
To Analysis	Затверджено потребу в реалізації. Задача передається аналітику для збору вимог та деталізації.
Analysis	Ведеться активне проектування, опис Use Cases, створення макетів та схем.
To Development	Критерії готовності до розробки виконані: вимоги задокументовані, погоджені РО, технічна оцінка внесена.
Development	Задача в роботі у розробника/дизайнера. Створюється програмний код.
To Testing	Код написаний, перевірений (Code Review), розгорнутий на тестовому середовищі. Очікує черги на тестування.
Testing	QA-інженер виконує верифікацію на відповідність Критеріям Прийняття.
For Release testing	Тестування успішно пройдено. Задача очікує включення в реліз та збірку.
Done	Реліз успішно задепloysено на Production, моніторинг завершено, зауваження відсутні.

Правила циклу доопрацювання (Rework Loop):

- Якщо під час статусу **Testing** виявлено баги, QA-інженер створює дочірні підзадачі (Sub-tasks) типу **Bug** із детальним описом кроків відтворення.
- Основна задача повертається у статус **Development** (або **Fix in progress** залежно від налаштувань проєкту).
- Після виправлення дефектів розробником, задача знову переводиться у статус **Testing** для повторної верифікації. Цикл повторюється до повного усунення критичних дефектів.

4. Спринт-процес (Sprint Process)

4.1. Планування спринту (Sprint Planning)

Проводиться у перший день спринту. Тривалість — до 2 годин.

1. Вхідні дані: Беклог Продукту, орієнтовна швидкість команди (Velocity) за минулі спринти, цілі спринту від Власника Продукту.
2. Процес:
 - Обговорення задач зі статусу **To Development**.
 - Оцінювання завдань командою розробки в годинах (Original Estimate).
 - На основі доступної ємності команди (Capacity) формується Спринт Беклог (Sprint Backlog).
 - Формулюється чітка Ціль Спринту (Sprint Goal).
3. Результат: зафіксований обсяг задач у спринті, підтверджений зобов'язанням (commitment) команди.

4.2. Виконання спринту (Sprint Execution)

- Daily Stand-up: щоденна синхронізація тривалістю до 15 хвилин. Кожен виконавець звітує: що зроблено вчора, що планується сьогодні, які є проблеми/блокери. РМ фіксує блокери та вживає заходів для їх усунення.
- Логування часу (Worklog): кожен виконавець зобов'язаний щоденно списувати фактично витрачений час на виконання задач у JIRA.

4.3. Огляд результатів спринту (Sprint Review / Demo)

Проводиться в останній день спринту. Команда демонструє Власнику Продукту та зацікавленим стейкхолдерам готовий інкрементований функціонал безпосередньо на тестовому або демонстраційному стенді. Презентація слайдів замість демонстрації працюючого коду не допускається. Власник Продукту приймає завдання, які відповідають критеріям прийняття. Неприйняті задачі переносяться в наступний спринт або повертаються в беклог.

4.4. Ретроспектива спринту (Sprint Retrospective)

Проводиться після Sprint Review. Мета — аналіз внутрішніх процесів, інструментів та взаємодії. Результатом є перелік конкретних дій з покращення (Action Items) із визначеними власниками та термінами виконання для наступного спринту.

5. Реліз-менеджмент (Release Management)

Реліз-менеджмент ДП «ІСС» спрямований на забезпечення стабільності та безпеки інформаційних систем судової влади при розгортанні змін.

5.1. Формування переліку задач для релізу

Керівник Проєкту спільно з Власником Продукту формують релізний пул.

- За замовчуванням до релізу включаються всі задачі, які на момент завершення спринту перебувають у статусі **For Release testing**, та виправлені дефекти (Bugs) у статусі **Done**.
- Завдання, що мають незакриті підзадачі або блокуючі залежності, виключаються з релізу та переносяться на наступний цикл.

5.2. Задача типу Release Deployment

Для кожного випуску створюється окрема задача типу **Release Deployment**, яка містить:

- Версію релізу відповідно до семантичного версіонування (наприклад, **v1.4.0**).
- План розгортання (Deployment Plan) та перелік відповідальних інженерів.
- Перелік пов'язаних задач JIRA, що входять до цієї версії.
- Посилання на підписаний QA-інженером звіт про тестування (Test Report) та Release

Notes.

5.3. Процес розгортання та моніторингу

1. Тестування на Staging: збірка розгортається на інтеграційному середовищі (Staging) для проведення регресійного та навантажувального тестування.
2. Production Deployment: після отримання погодження від СТО та РО, здійснюється розгортання на промисловому середовищі (Production) відповідно до затвердженого вікна обслуговування (maintenance window).
3. Моніторинг: після деплою встановлюється період підвищеної уваги (від 1 до 5 робочих днів). У разі виявлення критичних помилок терміново формується Hotfix.
4. Завершення: після успішного моніторингу задача **Release Deployment** переводиться в статус **Done**, що автоматично переводить усі пов'язані задачі з **For Release testing** у статус **Done**.

6. Управління завданнями та звітність (Reporting)

6.1. Відстеження прогресу й трудовитрат

Для цілей фінансового та технічного аудиту впроваджено жорсткі правила обліку часу:

- Original Estimate (Первинна оцінка): вноситься командою під час планування спринту. Забороняється змінювати це поле після старту спринту. Це забезпечує збереження базової лінії (baseline) для оцінки точності планування.
- Worklog (Журнал робіт): фактично витрачений час логується виконавцями щодня з точністю до 15 хвилин.
- Remaining Estimate (Залишок оцінки): розраховується автоматично або коригується вручну розробником, якщо під час виконання виявлено приховану складність.

6.2. Збирання та візуалізація даних в Apache Superset

Дані з JIRA (Worklogs, статуси, виконавці, дати створення/закриття) щоденно вивантажуються через ETL-процеси у базу даних аналітики. В Apache Superset налаштовані автоматичні дашборди для керівництва ДП «ІСС» та зовнішніх аудиторів. Звіти будуються за проектами, епіками, виконавцями та ролями.

7. Правила оформлення та ведення документації

Усі проєкти ДП «ІСС» повинні мати актуальну технічну документацію. Відсутність оновлень документації прирівнюється до невиконання вимог задачі.

7.1. Документація вимог (Use Cases / Specifications)

Для кожної бізнес-функції створюється Use Case у Confluence за затвердженим шаблоном:

1. Назва та унікальний ідентифікатор Use Case.
2. Актори (користувачі, зовнішні системи).
3. Передумови (Pre-conditions) та Післяумови (Post-conditions).
4. Основний сценарій взаємодії (кроки користувача та відповіді системи).
5. Альтернативні та негативні сценарії (обробка помилок, відсутність зв'язку).
6. BPMN-діаграма для візуалізації складних процесів.

Кожне завдання в JIRA повинно містити пряме посилання на відповідний Use Case.

7.2. Release Notes

Створюється для кожного релізу. Містить перелік нових функцій, виправлених багів, опис міграції баз даних та інструкції для адміністраторів щодо розгортання.

7.3. Технічна документація (Technical Architecture)

Описує архітектуру системи відповідно до стандартів TOGAF та ISO/IEC/IEEE 42010. Вона включає:

- Схеми компонентів та розгортання (Deployment Diagrams).
- Специфікацію API методів (запити, відповіді, коди помилок) у форматі OpenAPI/Swagger.
- Схеми структур даних (ER-діаграми).

Внесення змін до коду без відображення в технічній документації заборонено.

8. Безперервне вдосконалення процесу (Continuous Improvement)

8.1. Регулярні ревізії процесу

Не рідше ніж один раз на квартал Департамент управління проєктами разом із Архітектурним комітетом проводить ревізію діючих процесів. Аналізується статистика точності оцінок,

швидкість усунення дефектів та зворотний зв'язок від команд.

8.2. Процедура внесення змін до Регламенту

Будь-який член команди розробки має право ініціювати зміни до цього Регламенту через пропозицію на Retrospective. Пропозиція розглядається РМО та СТО. У разі затвердження, зміни вносяться до цього документа, оновлюється номер версії, а оновлений регламент публікується в Confluence з обов'язковим Teams-оповіщенням для всіх команд.

9. Впровадження регламенту та контроль (Implementation & Controls)

Контроль за дотриманням цього Регламенту покладається на Керівників Проєктів та внутрішню службу аудиту якості ДП «ІСС». Основні контрольні точки аудиту (Audit Checkpoints) включають:

1. Перевірку відповідності архітектурних описів стандарту ISO/IEC/IEEE 42010:2022 перед затвердженням ТЗ.
2. Перевірку наявності Use Cases та Критеріїв Прийняття у кожній задачі перед переведенням у статус **To Development**.
3. Контроль відповідності розгортання релізів вимогам безпеки та наявності підписаних Test Reports.
4. Перевірку повноти заповнення Worklog у JIRA на щотижневій основі.

10. Пропоновані аналітичні панелі в Apache Superset

Для моніторингу ефективності процесів та надання звітів аудиторам в Apache Superset розгортаються наступні візуальні панелі:

1. Burndown / Burnup Chart (Діаграма згорання/накопичення): відображає прогрес виконання годин у межах спринту. Дозволяє виявити ризик зриву цілей спринту на ранніх етапах.
2. Sprint Velocity / Capacity Chart (Швидкість та ємність команди): показує динаміку виконання годин від спринту до спринту, допомагаючи точніше планувати майбутнє завантаження.
3. Planned vs Actual per Sprint (Заплановані vs Фактичні години): порівнює сумарний Original Estimate із залогованим часом (Worklog) після завершення спринту. Використовується для оцінки точності планування.
4. Worklog & Original Estimate Accuracy (Точність оцінки задач): детальний звіт по задачах із найбільшим відхиленням фактичних витрат від початкової оцінки для подальшого аналізу на ретроспективі.

5. Task Breakdown by Status / Epic (Розподіл задач за статусами/епіками): показує поточну концентрацію трудовитрат та виявляє «вузькі місця» (наприклад, накопичення задач у статусі **To Testing**).
6. Bug Trend & Bug Severity (Динаміка дефектів): аналізує кількість відкритих/закритих багів, їх критичність та середній час усунення дефекту (Lead Time for Bugs).
7. Release Status Dashboard (Стан релізів): відображає готовність наступного релізу, показуючи частку задач у статусі **For Release testing** та кількість відомих дефектів.
8. Work Distribution by Roles (Розподіл робіт за ролями): показує баланс завантаженості між аналітиками, розробниками, дизайнерами та тестувальниками.
9. Lead Time / Cycle Time Analysis (Аналіз часу виконання): вимірює середній час проходження задачі від створення до статусу **Done** (Lead Time) та від старту розробки до готовності (Cycle Time).
10. Theme Dashboard (Дашборд стратегічних тем): агрегує витрати часу та прогрес виконання у розрізі глобальних стратегічних ініціатив підприємства.

Література

- [1] The Open Group. The TOGAF Standard, 10th Edition. The Open Group, 2022.
- [2] ISO/IEC/IEEE 42010:2022. Systems and software engineering — Architecture description. International Organization for Standardization, 2022.
- [3] ISO/IEC/IEEE 12207:2017. Systems and software engineering — Software life cycle processes. IEEE, 2017.
- [4] ISO 21502:2020. Project, programme and portfolio management — Guidance on project management. International Organization for Standardization, 2020.
- [5] ISO 9001:2015. Quality management systems — Requirements. International Organization for Standardization, 2015.
- [6] ДСТУ 3973-2000. Система розроблення та постачання продукції на виробництво. Правила виконання науково-дослідних робіт. Загальні положення. Держстандарт України, 2000.
- [7] ДСТУ 3974-2000. Система розроблення та постачання продукції на виробництво. Правила виконання науково-конструкторських робіт. Загальні положення. Держстандарт України, 2000.
- [8] Ken Schwaber and Jeff Sutherland. The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Play. Scrum.Org, 2020.
- [9] Project Management Institute. A Guide to the Project Management Body of Knowledge (PMBOK Guide), Seventh Edition. Project Management Institute, 2021.
- [10] ISO 15489-1:2016. Information and documentation — Records management — Part 1: Concepts and principles. International Organization for Standardization, 2016.